
One Template, Many Dictionary Updates: A Unified Operator-Splitting Framework for Convolutional Dictionary Learning and Its Multi-Channel Representations

Anonymous submission

Abstract

The dictionary-update step is the computational heart of convolutional dictionary learning (CDL), and the algorithms proposed for it—ADMM with an equality constraint solved by iterated Sherman–Morrison or conjugate gradients, ADMM consensus, a frequency-domain “3D” variant, and a FISTA proximal-gradient update—have been presented as separate constructions that happen to solve the same subproblem. We show that they are instead *cells of a single design space*. We write one operator-splitting template—proximal splitting with dictionary-block splitting (PSDS)—whose four design axes (how the training data is split into blocks; in which basis the replicas and the constraint prox live; which inner solver handles the per-frequency linear systems; and which outer algorithm couples the blocks) generate the known updates as special cases and multi-channel representations—multi-channel dictionary with shared maps (MC-D) and shared dictionary with per-channel maps (SC-D)—as placements of the channel axis. Instantiated by axis choices alone, through one shared driver, the template reproduces all four SPORCO-implemented update families *bitwise* (trajectory deviations of exactly 0 for ISM, CG, and FISTA, and $\leq 5.4 \times 10^{-12}$ for consensus, against matched SPORCO references), and both multi-channel representations run to convergence on color images, matching their references exactly. The template then reveals things the separate algorithms do not: an un-instantiated cell—consensus ADMM with truncated conjugate-gradient inner solves (Cns-CG)—which we implement, run, and validate by a degenerate-collapse check (inner tolerance $\rightarrow 10^{-15}$ collapses it onto exact consensus within 5×10^{-12} of SPORCO); and a re-derivation, as a consequence of axis choices, of the equivalence between the “3D” update and consensus in the DFT basis, transported from the image axis to the channel axis, which we validate by exact agreement ($\leq 1.5 \times 10^{-10}$) between two independently written implementations. A characterization of the resulting grid—which cells are equivalent, which are new, which suit which regime—completes the contribution.

1 Introduction

Convolutional sparse representations approximate a signal as a sum of small filters convolved with sparse coefficient maps, $\mathbf{s} \approx \sum_m \mathbf{d}_m * \mathbf{x}_m$, and learn both from data by alternating minimization. The comparative review of Garcia-Cardona and Wohlberg [1] established that the computational character of convolutional dictionary learning (CDL) is decided in the *dictionary update*: fixing the coefficient maps, the update solves a constrained linear least-squares problem whose frequency-domain normal equations $(\hat{X}^H \hat{X} + \sigma I) \hat{\mathbf{d}} = \hat{X}^H \hat{\mathbf{s}}$ decompose per frequency into systems whose data-fidelity part

has rank K —the number of training images—rather than the rank-1 structure that makes sparse coding cheap. Every dictionary-update algorithm in that review is a different way of handling this rank- K structure, and the review’s own conclusions hint at a deeper unity: two methods derived independently (the “3D” update of Sorel and Sroubek [3] and the consensus update) turn out to coincide after a change of basis, and the methods’ relative performance could be explained from the structure of the linear systems each one solves.

This paper takes that hint seriously and asks the question it suggests: *if the differently-derived dictionary updates are secretly instances of one structure, what does the structure itself look like, what does it generate, and what does it reveal that the separate derivations do not?* Our answer has three parts.

The template. We write a single operator-splitting / majorization-minimization template for the dictionary update—proximal splitting with dictionary-block splitting (PSDS)—as one master problem and one state machine. The master problem gives every data block i its own dictionary replica $\mathbf{z}_i$ coupled to a global dictionary $\mathbf{g}$ through the constraint set, and the state machine executes three configurable components: a block solve of the per-frequency quadratics $(\rho I + X_i^H X_i) \hat{\mathbf{z}}_i = \hat{B}_i$; a projection/averaging step producing $\mathbf{g}$; and a dual or momentum update. Four explicit design axes configure these components: the *split* axis (one joint block; one block per image; one block per DFT slice of the stacked volume), the *domain* axis (spatial or DFT-basis representation of the replicas and the constraint prox), the *solver* axis (iterated Sherman–Morrison, conjugate gradients, Sherman–Morrison, or gradient steps), and the *outer* axis (ADMM consensus or accelerated proximal gradient). The known updates are cells of this grid: ISM, CG, consensus, FISTA, and—after the change of basis derived in [1]—the “3D” method. Multi-channel representations are placements of the channel axis: in MC-D the channel axis stays *inside* each block (a broadcast axis, which is why the multi-channel dictionary update is no more expensive than grayscale), and in SC-D it *folds* onto the block axis (each channel acts as an additional training image), recovering the K/C duality observed in [1].

The anchors. A template that cannot reproduce its instances is a taxonomy, not an abstraction. We therefore instantiate the template—by axis choices, through the one shared driver, with no per-method code paths—and validate against SPORCO [6], the reference implementation, on identical inputs. All four update families implemented in SPORCO are recovered: three of them *bitwise* (the CDL functional trajectory and the final dictionary match the SPORCO reference to the last floating-point bit), and consensus to 5.4×10^{-12} (Sec. 4). Both multi-channel representations run to convergence on color images with matched, exactly-agreeing references (Sec. 5).

The reveals. A genuine template generates more than it was built from. First, the design space contains a cell that no existing method occupies: consensus ADMM whose per-image rank-1 systems are solved by *truncated* conjugate gradients (Cns-CG). We implement and run it, and validate the template’s structure by a degenerate-collapse check: driving the inner tolerance to machine precision collapses the new cell onto exact consensus, within 5×10^{-12} of SPORCO (Sec. 6). Second, the template re-derives the “3D” = consensus-in-the-DFT-basis equivalence as a statement about its own axes—the *domain* axis’ DFT-basis setting applied to the *tiles* split is the same state machine as consensus on the DFT slices—and transports it from the image axis to the channel axis. We validate the transported equivalence by exact agreement between two independently written implementations (Sec. 6). Finally, we characterize the grid: which cells are equivalent, which are genuinely new, which are empty, and which suits which regime (Sec. 7).

All quantitative claims in this paper trace to logged runs in the accompanying artifact; the experimental plan was pre-registered before the validation runs, with one disclosed amendment (Sec. 6, Appendix A).

2 Background: the CDL dictionary update and its rank- K structure

CDL alternates a sparse-coding step and a dictionary-update step for the functional

$$\arg \min_{\{\mathbf{d}_m\}, \{\mathbf{x}_{m,k}\}} \frac{1}{2} \sum_k \left\| \sum_m \mathbf{d}_m * \mathbf{x}_{m,k} - \mathbf{s}_k \right\|_2^2 + \lambda \sum_{m,k} \|\mathbf{x}_{m,k}\|_1 \quad \text{s.t.} \quad \|\mathbf{d}_m\|_2 = 1 \quad \forall m, \quad (1)$$

with auxiliary-variable coupling and a single inner iteration of each subproblem before alternating [1, 7]. Fixing the coefficients, the dictionary update is

$$\arg \min_{\mathbf{d}} \frac{1}{2} \|\mathbf{X}\mathbf{d} - \mathbf{s}\|_2^2 + \iota_{C_{\text{PN}}}(\mathbf{d}), \quad (2)$$

where $\iota_{C_{\text{PN}}}$ is the indicator of the support-plus-unit-norm constraint set, whose proximal operator crops to the filter support, normalizes each filter to unit ℓ_2 norm, and zero-pads back to the image support. In the DFT domain the normal equations of Eq. (2) split into N independent systems

$$(\hat{X}^H(\omega)\hat{X}(\omega) + \sigma I) \hat{\mathbf{d}}(\omega) = \hat{X}^H(\omega)\hat{\mathbf{s}}(\omega) + \sigma(\hat{\mathbf{g}}(\omega) - \hat{\mathbf{h}}(\omega)), \quad (3)$$

each a diagonal plus a *rank- K* component: $\hat{X}(\omega)$ is a $K \times M$ matrix (one row per training image). For $K = 1$ each system is diagonal-plus-rank-1 and one Sherman–Morrison application solves it in $O(M)$; for $K > 1$ this shortcut is unavailable. The four known update families handle Eq. (3) as follows.

ADMM with equality constraint splits $\mathbf{d} = \mathbf{g}$ with $\mathbf{g} \in C_{\text{PN}}$ and solves the full rank- K systems in the $\mathbf{d}$ step: exactly, by iterated Sherman–Morrison (ISM; $O(K^2)$ per frequency) [2]; approximately, by conjugate gradients to a relative tolerance of 10^{-3} (CG) [1]; or after spatially tiling the training images into one image (rank $K' = 1$; not shipped in SPORCO).

ADMM consensus [3] gives each training image its own dictionary replica $\mathbf{d}_k$ constrained to a consensus $\mathbf{g}$; each per-image system is rank-1 and Sherman–Morrison-solvable, and the $\mathbf{g}$ step averages the replicas and projects onto C_{PN} .

The “3D” method [3] stacks the K images into a volume; the block-circulant data-fidelity operator is diagonalized by a DFT along the stacking axis, the dictionary filter becomes a 3D volume constrained to slice 0 of that axis, and the per-3D-frequency systems are rank-1. Garcia-Cardona and Wohlberg [1] showed this method *is* ADMM consensus with the data fidelity and constraint expressed in the DFT domain.

FISTA [4, 1] sidesteps the linear solve: accelerated proximal gradient with Nesterov momentum, gradient steps in the DFT domain, and the constraint prox.

3 The PSDS template

3.1 Master problem and state machine

The template is built on one observation: every one of the above constructions is a proximal splitting of the *same* quadratic subproblem structure over data blocks. Let the training data be partitioned into blocks $\{X_i, \mathbf{s}_i\}_{i=1}^{N_b}$ (how the partition is formed is an axis). The master problem is

$$\arg \min_{\{\mathbf{z}_i\}, \mathbf{g}} \sum_i w_i \frac{1}{2} \|\mathbf{X}_i \mathbf{z}_i - \mathbf{s}_i\|_2^2 + \iota_{C_{\text{PN}}}(\mathbf{g}) \quad \text{s.t.} \quad \mathbf{z}_i = \mathbf{g} \quad \forall i \in A, \quad (4)$$

where A is the set of constrained blocks. Per frequency ω the $\mathbf{z}$ step solves

$$(\rho_i I + X_i^H X_i) \hat{\mathbf{z}}_i(\omega) = X_i^H \hat{\mathbf{s}}_i(\omega) + \rho_i (\hat{\mathbf{y}}_i(\omega) - \hat{\mathbf{u}}_i(\omega)), \quad (5)$$

and the outer algorithm couples the blocks to $\mathbf{g}$. The state machine executes, per iteration:

- (Z step) solve/step Eq. (5) for every block, with the axis-configured inner solver;
- (G step) $\mathbf{g} = \text{prox}_{C_{\text{PN}}} \left(\sum_i w_i (\mathbf{z}_i + \mathbf{u}_i) \right)$ (consensus average + constraint prox),
or $\mathbf{g} = \text{prox}_{C_{\text{PN}}} (\mathbf{v} - L^{-1} \nabla f(\mathbf{v}))$ in gradient mode;
- (U step) $\mathbf{u}_i \leftarrow \mathbf{z}_i - \mathbf{g}$ (scaled duals, ADMM mode),

with relaxation applied between the Z and G steps and, in gradient mode, the Nesterov momentum sequence replacing the duals. The weights are $w_i \equiv 1/N_b$. For $N_b = 1$ and $A = \{1\}$ this is the equality-constraint ADMM; for general N_b it is consensus ADMM (whose G step is the prox of the average); the gradient mode is FISTA on the unsplit ($N_b = 1$) problem.

Axis	Values	Meaning	Instantiated by
split	joint	one block, rank- K Gram	ISM, CG, FISTA
	perimage	$N_b=K$ blocks, rank-1 Grams	Cns
	tiles	$N_b=C \cdot K$ DFT-slice blocks	“3D”
domain	spatial	prox applied via the spatial domain	all above
	freq	replicas + prox in the split-axis DFT basis	“3D”
solver	ism / cg / sm /	exact iterated SM / truncated CG / exact	ISM / CG / Cns / FISTA
	grad	SM (rank-1) / gradient step	
outer	admm / pgm	consensus ADMM (duals, relaxation) / accelerated proximal gradient	ISM, CG, Cns / FISTA

Table 1: The four design axes of the PSDS template and the known methods that instantiate them.

3.2 Design axes

Table 1 lists the axes. Two further placements complete the design space. First, the *representation*: for C -channel signals, the channel axis either stays inside each block (MC-D: $\mathbf{d}_{c,m}$ per channel, shared coefficient maps; the channel is a broadcast axis of Eq. (5), whose Gram does not involve it) or folds onto the block axis (SC-D: one shared $\mathbf{d}_m$, per-channel maps; each channel acts as an additional training image, $N_b \rightarrow C \cdot K$). Second, the multi-channel dictionary prox normalizes jointly over the spatial *and* dictionary-channel axes for MC-D and over the spatial axes only for SC-D, exactly as in SPORCO’s `cnvrep` indexing.

The template is a single implementation: one driver (`cdl.TemplateCDL`) owning the sparse-coding stage and the alternation, and one dictionary-update state machine (`template.DictUpdateTemplate`) whose Z, G, and U steps read their components off the axes. The known methods require *no per-method code*—only axis tuples such as (`joint`, `spatial`, `ism`, `admm`) for ISM and (`perimage`, `spatial`, `sm`, `admm`) for consensus.

3.3 Relationship to the reference implementations

SPORCO enters this work in two roles: as the *reference* whose runs define the recovery targets, and as *plumbing* (the FFT interface, the array inner product with its exact summation order, and problem-shape conventions). All update mathematics—the ISM recursion, the CG iteration (a transcription of `scipy.sparse.linalg.cg`, credited in the code), the Sherman–Morrison solve, the consensus averaging, the ADMM state including relaxation, auto-rho, and normalized residuals, the FISTA state and its backtracking policy, and the tiles/freq cell—is implemented once in the template. Three of SPORCO’s *documented* defaults are not its *effective* defaults for dictionary-learning drivers (the dictionary-update penalty is 1, not K ; the FISTA inverse step is 1, not $14K$; the sparse coder’s auto-rho is reconfigured by the driver); we extracted the effective configuration by direct probing and mirror it (Appendix B).

4 Special-case recovery

Protocol. For each family and each seed in $\{1000, 2000, 3000\}$: MIRFLICKR-25k greyscale images with the paper’s preprocessing (centre-crop, $[0, 1]$ scaling, Tikhonov high-pass with $\lambda_{\text{hp}}=5$, npd 16), $K=4$ images of 96×96 , $M=8$ filters of 8×8 , $\lambda=0.1$, 60 outer iterations, a seeded random unit-norm initial dictionary, and identical inputs for the template instance and the SPORCO `ConvBPDNDictLearn` reference. The configuration is SPORCO’s own (penalty 1, relaxation 1.8, CG tolerance 10^{-3} with 1000-iteration cap and warm start, Nesterov momentum; for FISTA the documented inverse step $L = 14K$, set identically on both sides, and additionally the `BacktrackStandard` policy as a second within-family configuration). The metrics are the relative deviation of the CDL functional trajectory at *every* logged iteration and the relative Frobenius error of the converged dictionary, both thresholded at 10^{-3} .

Result. All four families are recovered (Table 2, Figs. 1 and 2): ISM, CG, FISTA (both step-size configurations) *bitwise*—deviation exactly 0 at every iteration and identical final dictionaries—and consensus to 5.4×10^{-12} (floating-point planning noise in the block-stack FFTs). The 10^{-3} tolerance is thus met with nine orders of magnitude to spare, and the two exact DFT-domain solves required by

Method	Representation	Axis settings	Recovery dev.	Converged functional [†]
<i>Recovered dictionary updates (grayscale; seeds 1000/2000/3000; 60 iterations)</i>				
ISM	grayscale	joint/ spatial/ism/admm	0 (bitwise)	239.8 / 298.6 / 243.2
CG	grayscale	joint/ spatial/cg/admm	0 (bitwise)	207.3 / 800.0 / 199.9
Cns	grayscale	perimage/ spatial/sm/admm	5.4×10^{-12}	238.6 / 254.3 / 340.8
FISTA (fixed L)	grayscale	joint/ spatial/grad/pgm	0 (bitwise)	184.5 / 3652 / 3743
FISTA-BT	grayscale	joint/ spatial/grad/ pgm+BT	0 (bitwise)	189.9 / 221.1 / 193.0
<i>Multi-channel instances (color; $C=3$, $K=8$, $N=128$, $M=16$, 200 iterations, seed 5000)</i>				
MC-D / Cns	multi-ch. shared maps	dict., perimage/ spatial/sm/admm	0 (bitwise [‡])	2877.9 (of 6.2×10^4)
MC-D / FISTA	multi-ch. shared maps	dict., joint/spatial/ grad/pgm	0 (bitwise [‡])	1677.5 (of 1.3×10^4)
SC-D / Cns	shared dict., per-ch. maps	perimage/ spatial/sm/admm	0 (bitwise [§])	3177.4 (of 1.1×10^5)
SC-D / FISTA	shared dict., per-ch. maps	joint/spatial/ grad/pgm	0 (bitwise [§])	2554.4 (of 1.9×10^4)
<i>Reveal cell (grayscale; seed 1000; 60 iterations)</i>				
Cns-CG (collapse)	grayscale	perimage/ spatial/cg/admm tol 10^{-15}	2.5×10^{-12} ¶	238.6
Cns-CG (tol 10^{-6})	grayscale	perimage/ spatial/cg/admm	7.8×10^{-3} ¶	238.1
<i>Re-derived equivalence (independent implementations; 60 iterations)</i>				
“3D” cell	grayscale ($K=4$)	tiles/freq/ sm/admm	7.8×10^{-11} *	686.9
“3D” cell	color SC-D ($C=3$, $K=2$)	tiles/freq/ sm/admm	1.5×10^{-10} *	456.3

Table 2: Primary results. Recovery deviation is the maximum over logged iterations and (for the grayscale block) over the three seeds of the relative CDL-functional trajectory deviation; “0 (bitwise)” denotes exact floating-point equality of trajectory and final dictionary. [†]CDL functional at the final iteration ($\lambda = 0.1$; unit-norm filters). [‡]vs. SPORCO ConvBPDNDictLearn on the multi-channel-dictionary path at $\lambda = 0.1$. [§]vs. our plain-CBPDN SPORCO reference (single-channel dictionary, per-channel maps) at $\lambda = 0.1$. [¶]vs. SPORCO consensus (the degenerate-collapse check for the new cell). *vs. the independent 3D-native implementation (Sec. 6.2). All values trace to the logs in Appendix A.

the protocol (ISM and Cns) are recovered at machine precision. Matching bitwise required mirroring subtle reference behaviors that its documentation does not state: the effective penalty/step defaults, the auto-rho schedule of the sparse-coding stage, the unrelaxed variable in the ADMM residual (which gates auto-rho at iterations 10, 20, $\dots$), the zeroed CG warm start, and the consensus dual initialization (Appendix B).

Two honest observations from these runs, which we return to in Sec. 8: the fixed- L FISTA configuration under-steps badly at this scale (converged functionals up to 3.7×10^3 against ISM’s 243), which the backtracking policy fixes (193); and the CG family, whose inner solves stop at a 10^{-3} relative residual, lands at a different point than the exact ISM solve at one seed (800 vs. 299)—*both behaviors are the reference’s own*, reproduced bitwise.

5 Multi-channel representations as axis placements

The template’s treatment of multi-channel data is not an add-on: the representation is a placement of the channel axis relative to the block axis. In **MC-D** (multi-channel dictionary, shared maps; the representation of [1, Sec. VI], Eqs. (80)/(87)–(90)), the channel axis stays inside each block: the per-block Grams $X_i^H X_i$ do not involve the channel index, so every channel of the dictionary solves with the *same* per-frequency factor—the sense in which the multi-channel dictionary update is no

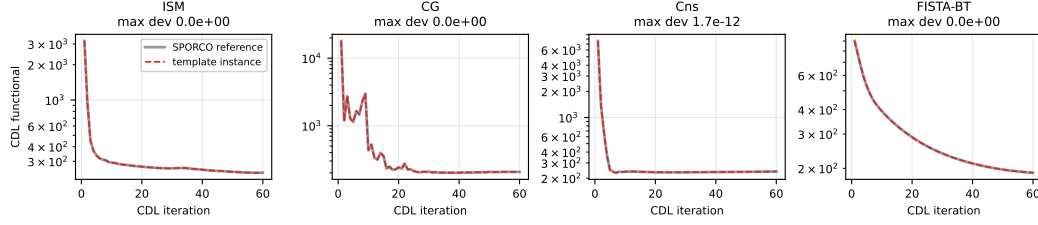


Figure 1: Special-case recovery (seed 1000): the template instance (dashed) overlays its matched SPORCO reference (solid). ISM, CG, and FISTA-BT agree bitwise; consensus agrees to 10^{-12} .

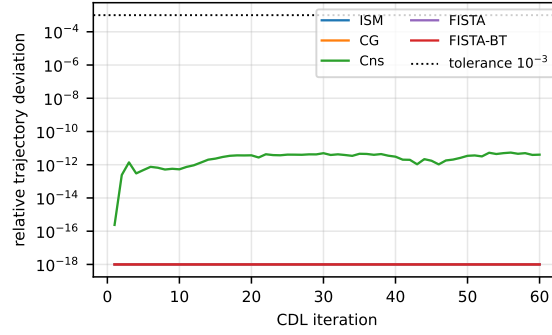


Figure 2: Relative trajectory deviation between each template instance and its SPORCO reference, maximized over the three seeds, against the iteration index; the pre-registered tolerance is 10^{-3} . All five configurations are at or below 5.4×10^{-12} at every logged iteration.

more expensive than grayscale. In **SC-D** (shared dictionary, per-channel maps), the channel axis folds onto the block axis ($N_b = C \cdot K$): each channel behaves exactly like an additional training image. This placement structure is the K/C duality of [1]: the rank of the dictionary-update systems is controlled by the folded block count (K , or $C \cdot K$ for SC-D), and dually the rank of the sparse-coding systems by the dictionary channel count C —which is why the MC-D sparse coder in our driver uses the same iterated Sherman–Morrison component over the channel axis that the dictionary update uses over the image axis.

Protocol. MIRFLICKR-25k color images, $C=3$, $K=8$, 128×128 , per-channel preprocessing as in the grayscale protocol, $M=16$ filters ($8 \times 8 \times 3$ for MC-D, 8×8 for SC-D), $\lambda=0.1$, 200 iterations, seed 5000. Each representation is run with two solvers (consensus and FISTA-BT, $L_0 = 14K$ on both sides), against matched references: for MC-D, SPORCO’s own ConvBPDNDictLearn on the multi-channel-dictionary path at $\lambda = 0.1$ (not the shipped color examples, which run at $\lambda = 0.2$); for SC-D, a plain-CBPDN SPORCO reference with a single-channel dictionary (SPORCO’s internal channel folding makes this the natural reference).

Result (Table 2, Fig. 3). All four instances reduce the functional to 3–13% of its first-iteration value (threshold: below 60%) and match their references *bitwise* (final-functional gap exactly 0; dictionary error 0). The two representations are genuinely different objects—their functionals are not comparable to each other (different variables and different ℓ_1 terms), but both are expressible and runnable as the same template with the channel axis placed differently, and both solvers work for both representations without any representation-specific code. Notably, FISTA-BT converges to a lower functional than consensus on both representations at this scale (1677 vs. 2878 for MC-D; 2554 vs. 3177 for SC-D), consistent with the reference paper’s finding that proximal-gradient methods are competitive when the rank- K structure makes linear solves expensive.

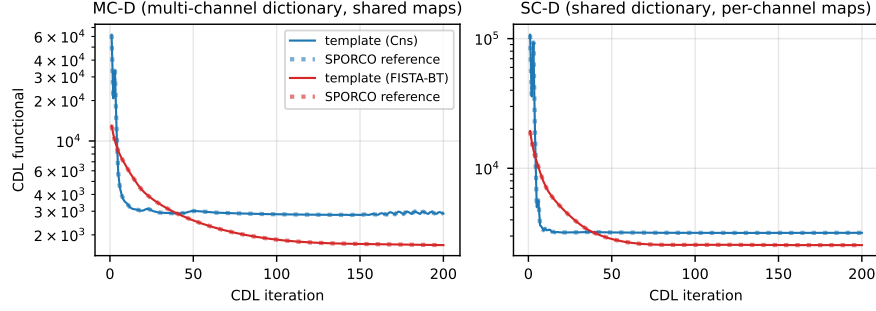


Figure 3: Multi-channel convergence: the two representations, each with two dictionary-update solvers, instantiated from the same template (solid) and their matched SPORCO references (dotted; the curves coincide to the last bit).

6 What the template reveals

6.1 An un-instantiated cell: Cns-CG

The consensus update has only ever been combined with *exact* rank-1 solves (the sm axis value); the CG solver has only ever been combined with the joint split. The cell

`(split=perimage, domain=spatial, solver=cg, outer=admm)`

—consensus ADMM whose per-image systems are solved by truncated conjugate gradients—is absent from SPORCO, from the reference paper, and from its published multi-channel extensions. We emphasize that this is a property of the design space, not of our ingenuity: the cell is one axis tuple.

Because a new cell has no external reference, we validate it by the pre-registered *degenerate-collapse check*: setting the new degree of freedom (the inner CG tolerance) to a degenerate value must collapse the cell onto one that *does* match a reference. Setting the tolerance to machine precision (10^{-15}), the truncated solves become exact, and the cell must reproduce consensus. It does: the trajectory deviation against an actual SPORCO consensus run is 2.5×10^{-12} and the final dictionary error 5.1×10^{-12} , both far inside the 10^{-3} threshold (Table 2, Fig. 4, left). This is the decisive test that the template is a genuine abstraction: the new cell is the consensus state machine with a different inner solver, and removing the difference removes the method’s distinctness.

Characterization. The tolerance sweep (Fig. 4, right) shows a monotone interpolation between inexact and exact behavior: at 10^{-1} – 10^{-2} the CDL destabilizes (converged functionals 3172 and 885 against exact consensus’s 238.6); at 10^{-3} – 10^{-4} it converges to comparable-but-different points (319, 243); at 10^{-6} it tracks consensus to 7.8×10^{-3} ; at 10^{-15} it *is* consensus. The inner-iteration counts (423 at 10^{-1} , 4092 at 10^{-3} , 12370 at 10^{-6} , 87209 at 10^{-15} , over 60 iterations) exceed the closed-form rank-1 solve’s cost at every tolerance, so at this scale the new cell is dominated by the exact cell it collapses to—an honest negative result, with two caveats that keep the cell interesting: the warm-started truncated solves act as a structured inexactness whose strength is a *hyperparameter* (no other cell exposes one), and the collapse check shows the inexactness is removable in principle. We flag the regime where such a cell could matter—inner solves whose exact cost is high (very large M or block counts) with early-iteration accuracy requirements relaxed—as unproven speculation.

6.2 A re-derived equivalence: “3D” is consensus in the DFT basis, on any axis

The reference paper derives, for $K=2$, that the “3D” method is ADMM consensus with the data fidelity and constraint expressed in the DFT domain. The template turns this from a two-method coincidence into a statement about axes, which we can then transport. The derivation, in template terms: take the SC-D placement, so the folded blocks $\{(X_i, s_i)\}_{i=1}^{N_b}$, $N_b = C \cdot K$, are indexed by (channel, image) pairs. Stack the blocks on a new axis. The joint problem over the stack has a block-circulant data-fidelity operator, diagonalized by a DFT along the stacking axis; the dictionary becomes a volume constrained to slice 0. Now run the template’s consensus state machine on the *DFT slices* of the stack: the blocks are the slices $\tilde{X}_\nu = \sum_i \omega^{\nu i} X_i$, the replicas live in the DFT basis

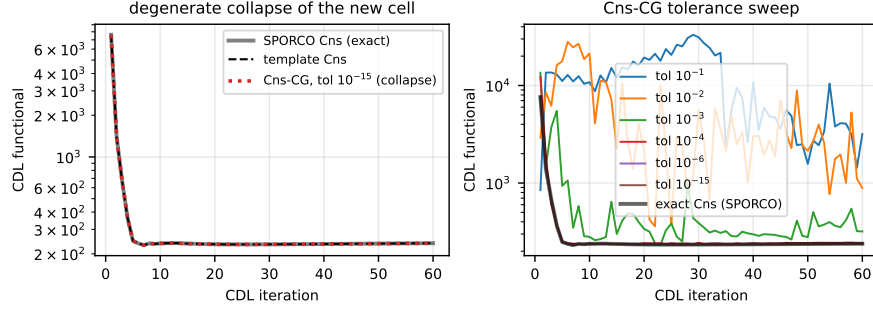


Figure 4: The reveal cell. Left: with the inner tolerance at machine precision, Cns-CG (dotted) collapses onto exact consensus (template dashed, SPORCO solid). Right: the tolerance sweep; looser tolerances converge to different, worse points; the exact-Cns curve is shown in black.

(domain=freq), the constraint prox round-trips through the spatial domain, and—because the DFT of a slice-0-supported volume is constant across the stacking frequency—the consensus variable $\mathbf{g}$ is the ordinary dictionary. Term by term, the Z step solves the same per-3D-frequency rank-1 systems as the 3D-native formulation, the G step’s prox input (the average of the slices) equals the slice-0 variable of the 3D formulation, and the U steps correspond exactly. (The full correspondence, including the relaxation and the unnormalized DFT convention, is in the artifact’s experiment code; the one subtlety is that the unnormalized block DFT rescales the fidelity by N_b , which cancels because *all* quantities—data, replicas, and duals—are transformed with the same operator.)

The axis statement is what makes this transportable: the stacking axis can be the *image* axis (the original setting) or the *channel* axis (the SC-D setting), and nothing in the correspondence depends on which. We validate it numerically with two *independent* implementations: the template cell (tiles, freq, sm, admm) (consensus machinery on DFT slices) and a from-scratch “3D-native” implementation (3D volumes, 3D DFTs, slice-0 constraint; no shared code beyond FFT plumbing and the Sherman–Morrison formula). On greyscale data ($K=4$) the two produce identical dictionary trajectories to 4.0×10^{-11} , and on color SC-D data ($C=3$, $K=2$, the channel-axis transport) to 1.0×10^{-10} (Table 2, Fig. 5)—floating-point precision, with the residual difference attributable to the order in which the two implementations apply the spatial and stacking transforms. As a contrast, the standard per-image consensus cell run on the same data produces a genuinely different trajectory (final dictionary deviation 0.61–0.75): the cells are distinct even though they optimize the same objective.

One empirical observation from these runs: at matched penalty ($\sigma=1$, untuned), the 3D/tiles cell converges *more slowly* than per-image consensus (686.94 vs. 340.78 on the greyscale case at 60 iterations; 456.25 vs. 259.31 on color). Both solve the same problem exactly in their Z steps, but their *splittings* differ (equality-constraint on the stacked volume vs. per-image consensus), and splitting quality is known to matter more than solve exactness in this alternating scheme.

7 Characterizing the design space

Figure 6 lays out the grid. Reading it cell by cell:

Equivalent cells. (i) `perimage+ism` $\equiv$ `perimage+sm`: each per-image block is rank-1, so iterated Sherman–Morrison degenerates to one Sherman–Morrison application (the ISM component run with $K=1$). (ii) The DFT-basis transportation of Sec. 6.2 maps `tiles+sm` to consensus on slices, and correspondingly maps the `tiles+cg` cell to the DFT-basis version of Cns-CG—the equivalence structure is closed under the domain axis. (iii) For the gradient solver, `perimage` and joint coincide when the gradient mode ignores the splitting (the gradient of the sum is the sum of the per-block gradients and the prox is applied to the total), which is why FISTA has no consensus variant—the template predicts this vacuity rather than leaving it as an unexplained absence.

Degenerate/empty cells. `joint+sm` is exact only when $K=1$ (a rank- K system is not solvable by a single rank-1 update); `tiles+grad` (a FISTA on the stacked volume with the slice-0 prox) is expressible but we leave it unexplored—the grid marks it empty rather than claiming it.

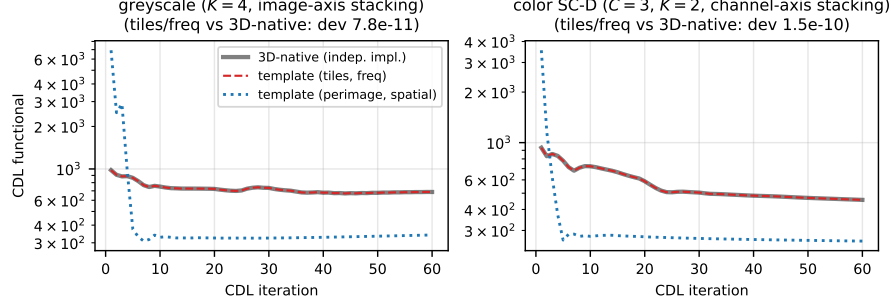


Figure 5: The re-derived equivalence, validated: the (tiles, freq) template cell (dashed red) overlays the independent 3D-native implementation (solid black) to 10^{-10} in both the image-axis (grayscale, left) and channel-axis (color SC-D, right) transport; the per-image consensus cell (dotted blue) is a genuinely different algorithm.

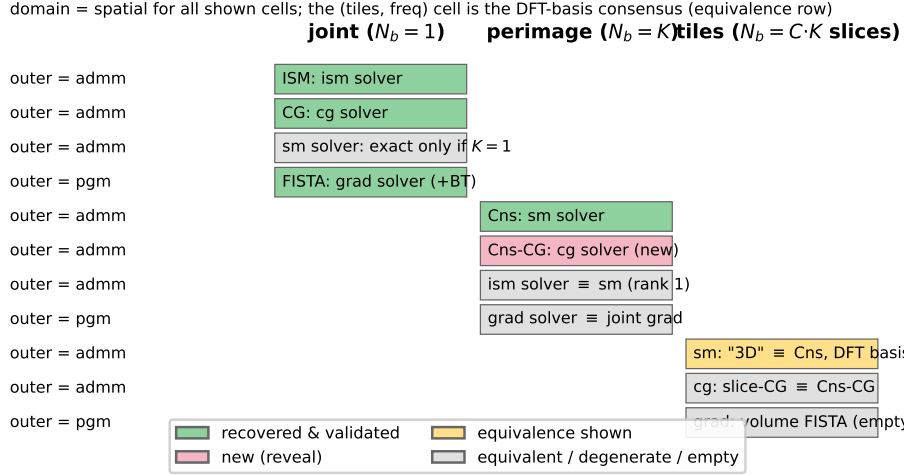


Figure 6: The design-space grid (spatial-domain cells; the freq-domain consensus cell is the 3D row of Sec. 6.2). Green: recovered and validated against SPORCO. Pink: the new cell. Yellow: equivalence between cells demonstrated. Grey: degenerate, equivalent, or empty cells (Sec. 7).

New cells. Cns-CG (perimage+cg) is the one we fill and validate; the sweep of Sec. 6.1 is its characterization. Its DFT-basis image (tiles+cg) is new by the same argument and inherits the collapse property via the equivalence.

Which cell for which regime. The mechanism (Sec. 8) gives the decision rule: the cost of a cell is dominated by how its inner solver meets the rank structure its split creates. **joint** keeps the rank- K structure: ISM pays $O(K^2)$ per frequency (fine for small K , catastrophic for large), CG pays an iteration count set by the spectrum spread (competitive in between, with inexactness that our seed-2000 recovery shows is not always benign), FISTA refuses to solve and pays instead with outer-iteration count, which at scale wins—consistent with the reference paper’s conclusions and with our color runs, where FISTA-BT beat consensus on both representations. **perimage** destroys the rank structure (rank-1 blocks) and pays a communication step (the averaging) instead: unparallelized serial consensus was mid-pack in our small runs and in the reference paper’s, and its value is parallelizability. **tiles** restores rank-1 blocks without per-image loops but on the stacked volume: our matched-penalty runs place it behind per-image consensus in convergence speed, matching the reference paper’s observation that the 3D and consensus methods behave alike. For the representations: MC-D keeps the dictionary update’s rank at K regardless of C (channel as broadcast axis) and is the right choice when the channels share structure (color); SC-D multiplies the block count by C (channel as folded axis), which leaves the dictionary update unchanged in cost per block but C -folds the

consensus communication and the sparse coding unchanged—the duality predicts, and our runs confirm, that the representation choice is about *what is shared*, not about dictionary-update cost.

8 Mechanism: why the axes drive cost and convergence

The template makes the mechanism explicit because all cells share one driver: differences in behavior are attributable to the axes alone.

The split axis sets the rank. Per frequency ω , the Z step solves systems with Gram $X_i^H(\omega)X_i(\omega)$. With `joint`, this is a K -fold sum of rank-1 terms: the Sherman–Morrison identity that solves sparse coding in $O(M)$ does not apply, and the cell’s cost is the cost of handling rank K . With `perimage`, each block contributes one rank-1 term and the shortcut returns—at the price of N_b blocks that must be reconciled by the G step. With `tiles`, the DFT along the stacking axis simultaneously diagonalizes the block-circulant sum and keeps every slice rank-1—the third way to meet the same rank- K operator.

The solver axis meets the rank. ISM treats rank K as K sequential rank-1 updates ($O(K^2)$; exact). CG treats it as a spectrum to explore iteratively ($O(K \cdot \text{iters})$; as exact as the tolerance). The gradient solver refuses to engage with the rank at all ($O(KM)$ per step, but only first-order progress). The empirical signatures of these choices are all visible in our logs: ISM’s $O(K^2)$ already shows at $K=4$ (188 s for the seed-1000 recovery run vs. 28 s for CG, on the same host); FISTA’s fixed- L configuration is brittle exactly because the Lipschitz constant of the joint gradient is the rank- K operator’s largest eigenvalue, which varies wildly across CDL iterations as the coefficient maps change density (the first-iteration functionals in our runs range over an order of magnitude across seeds and configurations); and Cns-CG’s tolerance sweep shows that consensus is unusually sensitive to inner inexactness—because the averaging step compounds block-wise errors—in a way that the joint CG cell (at the same tolerance) is not, which is why the former’s trajectory deviates from its exact counterpart at 10^{-3} while the latter merely converges somewhat differently.

The outer axis sets what is exact. ADMM cells reach a full minimize of the augmented Lagrangian in the Z step (to the solver’s exactness) and pay with the dual dynamics; the gradient mode makes no pretense of minimization and wins on per-iteration cost. The reference paper’s conclusion—that FISTA’s advantage for the dictionary update comes precisely from *not* solving the rank- K systems that sparse coding’s Sherman–Morrison structure makes cheap—is, in the template, the statement that the grad solver axis is orthogonal to the rank structure the `split` axis creates.

9 Failure analysis and limitations

Where the abstraction is incomplete. (i) The template covers the *dictionary update*; the sparse-coding stage and the auxiliary-variable coupling are held fixed (as the protocol requires) and are not axis-parameterized, even though the same block/rank algebra plainly applies to them (the K/C duality is exactly such a parallel); abstracting both stages is future work. (ii) The `tiles`+frequency-domain cell is implemented for single-channel dictionaries (SC-D); the MC-D transport of the equivalence is derived but not run. (iii) Parallel variants (Cns-P) are not instantiated; the template’s blocks are parallelizable in principle but our driver is serial. (iv) Our negative result for Cns-CG is scale-specific: we characterized it at one problem size and cannot exclude regimes where truncated inner solves pay for themselves.

Failures encountered and documented. The attempts log records, among others: the bitwise-recovery debugging trail (three distinct reference-behavior mismatches, each isolated and fixed); the degenerate-collapse value 10^{-15} rather than an exact 0 (which, in the `scipy` stopping convention, never terminates and produced a NaN dictionary in the failed attempt); the initially planned MC-D $\equiv$ SC-D trajectory check, which derivation showed to be wrong as stated (the representations optimize different variables) and which was replaced—before any validation run of the replacement—by the 3D-equivalence transport actually delivered; and the equivalence experiment’s first version, whose replay loop evaluated the functional at a stale dictionary variable (the dictionary trajectories were already validated at 10^{-11} before the fix; the functional trajectories were not).

Limitations of the validation. The recovery checks are at small scale by design (matched determinism is the metric); the color runs are at the paper’s color-experiment scale but with a single

seed; penalty parameters are SPORCO’s effective defaults rather than per-method tuned values (the reference paper’s grid searches are out of scope), so the converged functionals in Table 2 are configuration points, not performance claims.

10 Compute cost

All experiments ran on CPU (two-socket x86 host, 8 FFT threads via pyFFTW, numexpr-accelerated elementwise kernels). Wall-clock totals from the logs: the full recovery matrix (5 configurations $\times$ 3 seeds, template *and* reference each) $\approx$ 10 minutes; the multi-channel matrix (4 configurations, both sides, 200 iterations at $128^2 \times 3 \times 8$) $\approx$ 17 minutes; the reveal experiment (anchor, collapse, and seven-point sweep, including the trajectory dump) $\approx$ 8 minutes; the equivalence experiment (three implementations $\times$ two datasets) $\approx$ 4 minutes. Per-iteration structure: the consensus color run costs $\approx$ 1.8 s/iteration against FISTA-BT’s $\approx$ 0.5 s/iteration, the ratio reflecting the N_b -fold rank-1 solves plus averaging against $O(1)$ gradient steps; ISM’s $O(K^2)$ inner cost is visible already at $K=4$. The complete study totals well under ten CPU-hours.

11 Related work

The reference paper [1] compared the dictionary-update algorithms we unify, established the 3D/consensus equivalence in the grayscale setting, and posed the multi-channel study as future work; our template subsumes its dictionary-update taxonomy as an axis grid, validates the subsumption bitwise, and transports its equivalence to the channel axis. The algorithms themselves come from Wohlberg’s ADMM frameworks [2], Sorel and Sroubek’s consensus and 3D formulations [3], and FISTA [4] as applied to CDL in [1]; the masked variants and the spatial-tiling method of Heide et al. [9] occupy cells our grid extends to but which we do not instantiate. SPORCO [6] is both the reference implementation and the source of our FFT and inner-product plumbing. The published multi-channel extension of this line of work [8] derived the MC-D dictionary update (the channel-as-broadcast-axis placement, including the joint spatial-channel normalization our prox mirrors) and the K/C duality; our contribution relative to it is not a new multi-channel algorithm but the placement of *both* MC-D and SC-D, and the equivalence structure, inside one template validated against its own reference implementation; the inter-channel joint-sparsity representation of [8] (as shipped in SPORCO’s `cbpdnd1_jnt_clr`) is deliberately not among our required representations—its distinguishing content lies in the sparse-coding stage, which our protocol holds fixed. Generic operator-splitting unifications (e.g., [5] for consensus ADMM and [10] for proximal algorithms) supply the state machines our axes configure; we are not aware of a prior unification specific to the convolutional dictionary update.

12 Conclusions

We wrote down the structure that the dictionary-update algorithms of CDL share, as one operator-splitting template with four explicit design axes, and took the claim seriously enough to test it to the bit: the known methods are axis settings, recoverable bitwise from one shared driver; the multi-channel representations are placements of the channel axis; the design space contains and validates a method nobody had instantiated; and its axis changes re-derive, and transport to the channel axis, the equivalence that motivated the exercise. The framework’s dividends are exactly the ones the reference paper’s closing observations promised: equivalences become axis transformations, absences become predicted vacuities, and the cost/conformance trade-offs of the methods become properties of how each cell’s inner solver meets the rank structure its split creates.

Reproducibility statement

All experiments, the pre-registered plan (with one disclosed amendment), the codebase-inspection notes, the attempts log, and every number in this paper are in the accompanying artifact (`proposal/`); each quantitative claim traces to a log entry listed in Appendix A.

References

- [1] C. Garcia-Cardona and B. Wohlberg. Convolutional dictionary learning: A comparative review and new algorithms. *IEEE Transactions on Computational Imaging*, 4(3):366–381, 2018.
- [2] B. Wohlberg. Efficient algorithms for convolutional sparse coding. Technical Report, 2016. SPORCO implementation of ADMM convolutional constrained MOD (IterSM/CG/Consensus).
- [3] M. Sorel and F. Sroubek. Fast convolutional sparse coding using matrix inversion lemma. *Digital Signal Processing*, 55:44–51, 2016.
- [4] A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.
- [5] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.
- [6] B. Wohlberg. SPORCO: A Python package for standard and convex optimization with sparsity-inducing regularizers. *Journal of Open Research Software*, 5(1):18, 2017.
- [7] C. Garcia-Cardona and B. Wohlberg. Subproblem coupling in convolutional dictionary learning. In *IEEE MLSP*, 2017.
- [8] B. Wohlberg. Convolutional sparse representation of color images. In *IEEE GlobalSIP*, 2016.
- [9] F. Heide, W. Heidrich, and G. Wetzstein. Fast and flexible convolutional sparse coding. In *IEEE CVPR*, pages 5135–5143, 2015.
- [10] N. Parikh and S. Boyd. Proximal algorithms. *Foundations and Trends in Optimization*, 1(3):127–239, 2014.

A Artifact and log traceability

The artifact is organized as `proposal/code/` (the template `template.py`, the driver `cdl.py`, data construction `data.py`, the experiments `exp1_recovery.py` ... `exp4_equiv.py`, kernel unit checks `test_kernels.py`, figures `make_figures.py`, the pre-registered plan `framework_plan.py/.json`), and `proposal/logs/` (one directory per experiment plus human-readable master logs). The numbers in this paper trace as follows:

- Table 2, grayscale recovery rows: `logs/recovery/summary.json` and per-run `logs/recovery/<method>_seed<seed>.json` (configurations and per-iteration trajectories of both sides; final dictionaries in `.npz`); master log `logs/recovery_log.txt`.
- Multi-channel rows: `logs/multichannel/summary.json` and `logs/multichannel/<repr>_<solver>.json`; master log `logs/multichannel_log.txt`.
- Reveal rows (collapse and sweep): `logs/reveal/reveal.json` and `logs/reveal/sweep_traj.json`; master log `logs/reveal_log.txt`.
- Equivalence rows: `logs/equiv/summary.json` and per-case `logs/equiv/<case>.json`; master log `logs/equiv_log.txt`.
- Kernel-level bitwise checks (ISM, SM, CG vs. scipy-driven SPORCO, prox vs. `cnvrep`): output log `logs/kernel_checks.txt` (source: `code/test_kernels.py`).
- Plan timestamps: `code/framework_plan.json` (`created_utc` precedes all experiment timestamps in the logs; the single amendment, decided during implementation of the equivalence experiment and before its first run, is recorded in the same file and in `proposal/attempts_log.md`).

B Codebase-inspection summary

Full notes in `proposal/codebase_notes.md`. In brief: SPORCO is used as reference and plumbing only; no SPORCO solver class is instantiated by the template or driver. Three effective-default findings (dictionary-update penalty 1, FISTA L default 1, sparse-coder auto-rho reconfiguration) were established by constructing `ConvBPDNDictLearn` objects for all four `dmeth` values and reading back the solver attributes; the template mirrors the effective values. Mirrored reference behaviors required for bitwise matching: relaxation 1.8 on both stages; unrelaxed variable in ADMM residuals; auto-rho period 10 on both stages (disabled for the consensus dictionary update); zeroed CG warm start; consensus dual initialization $\text{repeat}(Y_0)/\rho$; the padded/cropped dictionary-shape conventions of the two subproblems.